

Consumer-Anchored Overlap for Collective-GEMM Execution

Anonymous Author(s)

摘要

张量并行层中的 AllGather、ReduceScatter 和 AllReduce 直接决定后续矩阵乘法或归约何时能够继续。若运行时以完整 rank、segment 或 panel 交接, consumer 会等待启动所不需要的其余数据, 通信等待和暂存占用因而进入关键路径。本文提出以 consumer 最小依赖为锚点的 collective-GEMM 重叠运行时: 它按可消费边界发布细粒度 ready, 以 frontier work 优先解除当前等待, 并以 streaming work 维持后续供给。对于 one-sided 路径, ready/free ticket 与活动窗口保护有限对称暂存槽位的安全复用, 使驻留规模受活动工作集而非逻辑工作总量约束。该协议实例化为 AG-GEMM、GEMM-RS 和 GEMM-AR, 并通过受形状和对称内存约束的有限测量选择配置。实验在 A100 和 K100_AI 上评估算子级性能与资源可行性, 并在 AG/RS 张量并行 prefill 路径上评估 Attention、MLP 和完整模型推理; AR 的评价限定为算子级。在已测工作负载上, 该协议缩短 collective-GEMM 链中的等待, 同时控制对称暂存驻留。

Keywords: collective-GEMM; overlap; tensor parallelism; NVSHMEM; Triton

1 引言

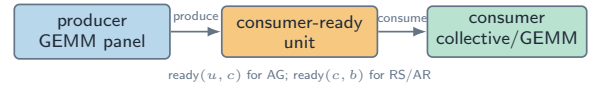
生成式模型的规模和上下文长度持续扩展。DeepSeek-V4、Kimi K3 和 Qwen3.8 已报告万亿级总参数配置 [6, 15, 32], 训练与推理系统因而以张量、流水线和数据并行分布模型状态与激活 [13, 17, 23, 27, 34, 44]。AllGather、ReduceScatter 和 AllReduce 会反复进入这些路径, 其通信成本及与计算的资源竞争会影响端到端效率 [1, 13, 18, 19]。

本文聚焦 collective 与 GEMM 由局部数据依赖连接形成的 producer-consumer 链。AG-GEMM

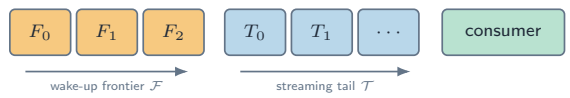
Consumer-anchored overlap runtime

minimum dependency $\rightarrow$ launch priority $\rightarrow$ bounded residency

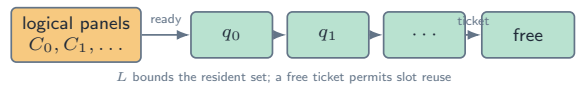
1. Dependency boundary



2. Frontier-first schedule



3. Bounded symmetric staging



4. Operator-specific ready units



ready before consume — free before reuse — every unit commits

图 1. 以 consumer-ready 为锚点的 collective-GEMM 重叠协议。

中, AllGather 逐步产生输入分片; GEMM-RS 中, ReduceScatter 消费 GEMM 的局部输出; GEMM-AR 中, AllReduce 聚合 GEMM 输出 [16, 23, 42]。若只在完整 rank、segment 或 panel 完成后交接, consumer 会等待远大于启动所需的数据集合。本文把满足 consumer 启动所需的最小依赖集合称为 consumer-ready; 它只表示数据可用, producer 的生产顺序和暂存资源仍决定 consumer 能否及时执行。

细粒度 ready 只暴露了执行资格, producer 还需优先完成能够解除当前等待的工作。图 2 比较三个 GEMM-RS shape 上 uniform 与 frontier-first 实现变体的链路时序; 所示记录中, frontier-first 变体从 producer launch 到首个 output commit 的总时长更

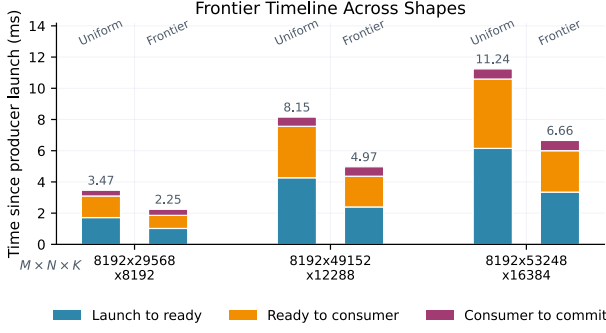


图 2. 三个矩阵 shape 上 GEMM-RS 的归档调度时序。

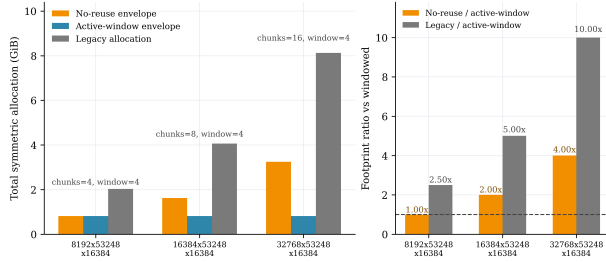


图 3. 活动窗口对称暂存的推导分配包络。

短。该比较是实现变体的链路记录，不是完整模型端到端时延，也不将差异完全归因于单一生产顺序。

更早暴露 ready 会增加 in-flight 工作，one-sided 路径还需限制 physical symmetric staging 的驻留规模。图 3 对比固定 $L = 4$ 的活动窗口、无复用和逐逻辑单元分配在逻辑 chunks 增长时的对称分配包络；窗口化映射保持有限 physical residency。该图由配置映射推导，不表示端到端时延或总 GPU memory。

已有工作分别通过任务分解、kernel/tile 编排、producer 完成事件或 collective 算法参数暴露重叠机会 [3, 5, 10, 12, 28, 33, 37, 38, 40, 43]。其中，FlashOverlap 已经证明 GEMM producer 的 tile/wave 完成信号可以在不打断主 GEMM 循环的情况下触发 collective，并用 wave grouping、pre/post reordering 和 predictive search 处理通信粒度、地址连续性与配置选择 [10]。本文的研究问题不是再次引入一个 producer signal，而是把 signal 的执行边界改写为 consumer-specific：只有足以解除某个 consumer 当前等待的最小依赖才进入 ready 状态；在此

基础上，producer 先执行能够关闭该等待的 frontier work，再用 tail work 维持流水线供给；对于 one-sided RS/AR，则继续以 arrival/free ticket 和活动窗口管理有限 symmetric staging 的安全复用。配置层面，本文先以 shape、窗口边界和 symmetric-memory 预算筛选不可行组合，再对少量代表性配置进行短测量，而不是把 wave-group predictor 作为运行时控制核心。本文将这一路线概括为 *consumer-anchored overlap*（消费端锚定式重叠），并与分解式、融合式、信号式和 collective 算法路线在表 1 中统一对比。

基于 Triton-distributed 的 Triton kernel 与 one-sided collective 基础 [26, 42]，本文作出三项贡献。

1. 提出以 consumer-ready 为一等控制对象的 collective-GEMM 重叠协议：将 consumer 的最小启动依赖显式化，并以 frontier-first 优先安排解除当前等待的 producer work、以 tail work 维持后续供给。该控制语义区别于仅依据 producer tile 完成来触发通信的信号式方案。
2. 设计 ready/free ticket 与活动窗口组成的 one-sided 暂存生命周期协议，将逻辑 panel/stripe 映射到有限 physical slots，并在 consumer 完成读取或归约后安全复用；在同一骨架下实例化 AG-GEMM、GEMM-RS 与 GEMM-AR。
3. 以形状、窗口和对称内存约束筛选不可行配置，并在 A100 与 K100_AI 上报告算子级、AG/RS prefill 与机制证据；AR 限于算子级，端到端集成限于 AG/RS 张量并行 prefill。

余下内容依次讨论相关工作、统一协议及其算子实例化、实验结果与适用边界。

2 背景与相关工作

本节按直接控制点而非发表时间组织相关工作：分解、融合、信号和 collective 运行时在不同层次暴露或调度重叠机会，其与本文的关系取决于依赖边界和资源状态。

2.1 分解式计算-通信重叠

分解式方法把计算、张量或通信拆为可调度单元，并在单元间安排通信以缩短串行区间。OCDC、Domino

与 CoCoNet 分别从依赖计算、通用张量切片和编译层跨抽象边界实施分解 [12, 37, 38]; Centauri、CO2、Oases、Async-TP 与 Syncopate 则将通信分区、训练交错或 chunk-centric 编排用于运行时配置 [4, 19, 30, 31, 35]。这些工作表明切分粒度、collective 原语和调度顺序会改变并发范围。

推理系统还将切分对象延伸到序列、token 和请求: ISO、TokenWeave、CommFuse 与 SARATHI 分别面向序列交错、token 任务、尾部时延和 chunked prefill [2, 8, 14, 39]。ConCCL 与 Characterizing Compute-Communication Overlap 则刻画 GPU DMA 并发下的资源竞争 [1, 18]。这些工作提供了任务分解和资源约束的背景, 但并不定义本文以 consumer 依赖为中心的 ready、frontier 与 staging 协议。

2.2 融合、生成与透明触发

融合、生成与透明触发路线把通信编排置于 kernel 附近。FLUX、TileLink 和 cuBLASmp 分别采用 kernel fusion、tile-centric primitive 与库级 GEMM/通信接口 [3, 24, 43]; T3 和 Comet 以计算事件触发 collective 或处理 MoE 的细粒度交错 [28, 41]。它们的控制点位于 kernel、tile、库接口或事件触发处, 与本文的协议层设计互补。

2.3 信号式重叠

信号式路线把 kernel 内部的局部完成事件变成后续通信的触发条件。FlashOverlap 是这一方向的代表: GEMM producer 在 tile 完成后更新计数状态, 通信侧以 tile 或 wave group 为粒度启动, 并通过 pre-communication reordering 形成 NCCL 可直接消费的连续地址、在通信后恢复逻辑顺序; wave grouping 的选择再由 predictive search 调整, 以平衡信号开销、通信带宽和重叠窗口 [10]。这一共同基点与本文一致, 即都利用 producer 的渐进进度而不是等待完整 GEMM 或完整 collective。

两者的直接控制对象并不相同。FlashOverlap 的触发条件是 producer tile/wave 已完成, 所回答的是“何时可以把已完成的数据交给通信库”; 本文将

ready 定义为某个具体 consumer 获得继续执行所需的最小依赖集合, 所回答的是“哪个 consumer 已经可以解除当前等待”。因此, 本文的 frontier-first 不是一般的 tile 或 wave 重排: 它先把 producer 资源投入当前 consumer-critical path 上的 wake-up panels, 再以 tail work 维持后续供给。该顺序与 ready 的语义绑定, 而不是由地址连续性或通信分组单独决定。

本文还把 one-sided 暂存的生命周期纳入同一协议。对于 GEMM-RS/AR, producer 在写入 peer-visible slot 前等待前序 free ticket, 完成后发布 arrival; consumer 收齐当前 panel/stripe 的依赖并完成读取或归约后发布 free, 活动窗口据此限制同时驻留的 physical slots。FlashOverlap 的所引段落集中于 wave signaling、NCCL handoff 与 pre/post re-ordering, 并未将这种 arrival/free ticket 与 bounded symmetric staging 作为显式控制对象。

2.4 Collective 算法与分布式运行时

TACCL、MSCCLang、Blink 和 AutoCCL 分别处理 collective 算法综合、描述语言、通用 collective 与 NCCL 参数调节 [5, 33, 36, 40]。它们优化算法、拓扑适配或库参数; 本文不改变 AllGather、ReduceScatter 或 AllReduce 的底层算法, 而在其上编排 consumer 依赖。

NCCL 和 NVSHMEM 提供 collective 与 GPU one-sided/symmetric-memory 基础 [25, 26]; Demystifying NCCL 和 Demystifying NVSHMEM 分析相应的通信与对称内存行为 [11, 21]。Triton-distributed 在 Triton kernel 上提供 one-sided AG、RS 和 AR 原语 [42], 本文在其上加入 consumer-ready、frontier-first 与 active-window; 后者只约束活动对称暂存的驻留包络。流水线/序列并行和 serving 的阶段划分会改变这类链路出现的位置与频率 [7, 9, 13, 16, 17, 20, 22, 23, 27, 29, 44], 但不替代链内的 ready、producer 排序或 staging 复用。

2.5 设计空间定位：从 producer 锚点到 consumer 锚点

上述路线可以按“重叠的锚点在哪里”统一刻画：分解式锚定可调度的切分单元，融合式锚定 kernel 与库接口边界，信号式锚定 producer 的局部完成事件，collective 算法工作锚定算法与库参数。这些代表性路线通常从 producer 侧的完成事件、kernel 或库接口边界以及 collective 控制点暴露重叠机会。本文将锚点移到 consumer 一侧，提出 *consumer-anchored overlap* (消费端锚定式重叠)：ready 显式表示某一 consumer 已集齐继续执行所需的最小依赖 (AG 的 row chunk、RS 的 chunk-band panel、AR 的 stripe)；frontier-first 依据该语义排序 producer 工作，优先关闭当前 consumer 的等待并以 tail work 维持供给；arrival/free ticket 与活动窗口进一步把有界 one-sided 暂存的安全复用纳入同一协议。表 1 按直接控制点、触发语义、调度目标与通信/暂存组织对四族路线与本文逐一对比。表中能力均以各文献所报告内容为准：某族未报告的控制对象不解释为该族方法不能实现，该表也不构成任何性能排名。

3 系统设计

AG-GEMM、GEMM-RS 和 GEMM-AR 的通信语义不同，但都包含同一关系：producer 逐步产生局部数据或结果，consumer 在依赖满足后执行。原始路径的空转来自三个相互制约的因素：可消费边界过粗、producer 未优先服务当前等待，以及为提前暴露的数据保留了随问题规模增长的对称暂存。本文分别以细粒度 ready、frontier/tail 编排和活动窗口处理这些约束。

3.1 概述 (Overview)

设并行组包含 p 个 ranks。 A_s, W_s, D_s 分别为 rank s 的 activation、weight 和 output shard，全局 GEMM 为 $M \times N \times K$ ，本地行数为 $m = M/p$ 。 p_ℓ 表示 peer-visible symmetric staging 域的 ranks 数；当前 one-sided 快路径在单节点 full-mesh NVLink 域内执行，故 $p_\ell = p$ 。 C_c, B_b 和 S_t 分别表示 row chunk、column band 和 AR stripe。

令 Ω 为一次调用的逻辑工作集合， $\mathcal{A}_r \subseteq \Omega$ 为尚未回收的活动集合。RS 以 L 个窗口 slot 复用远端可见 workspace Q_d ，其中 $q = c \bmod L$ ；AR 以 R 个 compact slots 承载 stripe， $r = \text{id}(c, b, t) \bmod R$ ，其中 id 是该 stripe 在本轮任务序列中的位置。表 2 汇总其余符号。

三类路径仅在 producer 与 consumer 的具体身份上不同：AG 逐步暴露远端输入供 GEMM 消费；RS 将局部 partial result 投递给 destination reduction；AR 使各 rank 生产并消费聚合结果。下文以同一数据流描述它们：ready 决定依赖何时可见，frontier/tail 决定 producer 的推进顺序，活动窗口限定已暴露工作的资源包络。

3.2 细粒度 Ready 单元与分叉式重叠 (Fine-Grained Ready Units and Bifurcated Overlap)

有效重叠要求 consumer 在 producer 尚未结束时获得足以启动的输入；因此 ready 单元应与自然消费边界对齐，而不是简单追求更细的切分。AG 的边界为可独立读取的 row chunk，RS 为可独立归约的 chunk-band panel，AR 以 panel 组织生产并以 stripe 完成物理聚合。过细的单元会引入 signaling、polling 和调度开销；即使数据已经可消费，producer 若优先计算不能解除当前阻塞的工作，consumer 的首次启动仍不会提前。

本文据此区分 wake-up work 与 streaming work：前者补齐当前 consumer 缺少的依赖，后者在流水线建立后维持供给。RS/AR 用 F 个优先 chunks 的 panels 构成 $\mathcal{F}$ ，其余 panels 构成 $\mathcal{T}$ ；frontier 是既有逻辑工作的执行顺序，而非新增阶段。AG 采用相同的分叉思想：local path 立即消费 A_s ，remote path 由远端 chunk-ready 逐步驱动。

运行时将 AG 的远端输入按 (u, c) 、RS/AR 的局部结果按 panel (c, b) 发布为 ready；consumer 只在所需输入或 contributions 到齐时执行。RS/AR 先推进 $\mathcal{F}$ 以尽早形成 panel-ready，再与 $\mathcal{T}$ 交错以持续供给。图 4 给出无重叠、ready 驱动和分叉式重叠的依赖形态。

表 1. 计算-通信重叠路线的机制定位：四族既有路线与本文 consumer-anchored 协议。能力描述以各文献报告内容为准，不表示性能排名。

路线（代表工作）	直接控制点与触发语义	调度目标	通信与暂存资源组织
分解式 [12, 30, 37, 38]	把计算、张量或通信切分为可调度单元，在单元间安排通信；触发以单元完成为准。	扩大并发范围、缩短串行区间；切分粒度与顺序由运行时配置决定。	沿用既有 collective 原语与其缓冲组织。
融合与透明触发式 [3, 28, 41, 43]	控制点位于 kernel 融合、tile 原语、库接口或计算事件触发处。	就地消除计算-通信边界，利用 kernel 内并发与访存局部性。	依赖 kernel epilogue 与库内部缓冲管理。
信号式 [10]	GEMM producer 在 tile/wave 完成后更新计数状态并发布信号，通信侧据此启动。	wave grouping 平衡信号粒度、带宽利用与重叠窗口，predictive search 选择分组。	pre/post reordering 形成 NCCL 可消费连续地址并在通信后恢复顺序。
Collective 算法与运行时 [5, 25, 26, 33, 42]	优化 collective 算法、拓扑映射或库参数。	降低 collective 自身时延、提高链路利用率。	提供 collective 与 one-sided/symmetric-memory 原语基础。
消费端锚定式（本文）	ready 显式表示某一 consumer 已集齐最小启动依赖：AG row chunk、RS chunk-band panel、AR stripe；触发语义由 consumer 侧依赖定义。	frontier-first 优先关闭当前 consumer 的等待，tail work 维持供给，缩短 consumer-critical path。	arrival/free ticket 将逻辑工作映射到有限 physical slots；活动窗口约束有界对称暂存并保证安全复用。

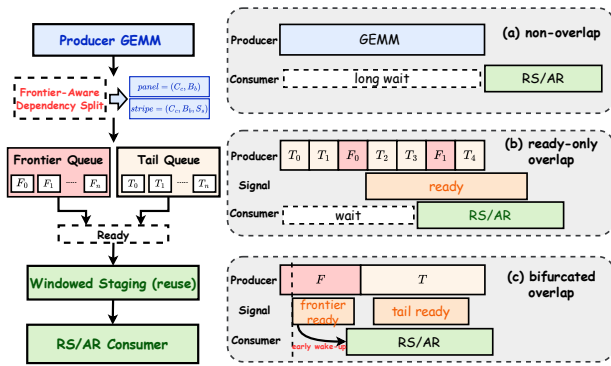


图 4. GEMM-RS/AR 的 panel-ready 依赖与 frontier-first 生产。

一次调用内，frontier 按调度顺序取前 F 个 chunks。 F 随 chunk 数、ready 粒度及通信/consumer 相对速度而变，因而作为与 shape 绑定的搜索参数，而非跨 shape 的固定常数。细粒度 ready 暴露可消费性，frontier/tail 编排决定这种可消费性何时转化为执行。

3.3 活动窗口对称暂存（Active-Windowed Symmetric Staging）

更早暴露 ready 和推进 frontier 会增加 in-flight 工作。RS/AR 需要把结果写入 consumer 可见的 symmetric memory [26]；若每个逻辑 chunk、band 或 stripe 永久

绑定物理区域，暂存会随逻辑工作集增长。slot 又只能在 consumer 完成读取和归约后复用，因此窗口过小会阻塞 producer，窗口过大则扩大内存包络。RS 的远端 staging 由活动 chunk-band window 决定，reducer stream 只影响归约节奏；AR 的 R 个 compact stripe slots 则直接构成 staging 容量。

本文把逻辑集合 Ω 与活动驻留集合 $\mathcal{A}_r \subseteq \Omega$ 分开，并用可复用映射 ϕ 将活动工作映射到有限 slots。对 RS，

$$\phi_{RS}(c) = q = c \bmod L. \quad (1)$$

这里 L 限制同时驻留的 chunks。每个 panel 携带单调递增 ticket：数据写入后发布 arrival，consumer 收齐当前 ticket 后读取并归约，归约完成才发布 free。因而 $arrival(t)$ 不早于对应数据写入，consumer 不在依赖 ticket 完整前读取， $free(t)$ 才允许下一轮覆盖同一 slot。这些不变量既避免陈旧信号匹配，也为 frontier-first 提供有界资源。

图 5 展示该循环映射。RS 中 source 在写入 $Q_d[q, b, s]$ 前等待前序 free ticket 与 chunk_signal，写入后发布 arrival；destination 等待本地 signal 和所需 arrivals，归约后发布 free。物理地址由此复用，而逻辑 ticket 保持严格配对。

图中沿用绘图文件的 Q_0, Q_1 标注来表示循环复用的物理窗口位置。为避免与 destination workspace

表 2. 方法部分的统一符号约定

符号	含义
p, p_ℓ	分别表示并行组 world size 和一个 peer-visible staging 域中的 local world size。
s, d, u	分别表示 source rank、destination rank 和 AG 中的远端 rank。
A_s, W_s, D_s	activation shard、weight shard 和 output shard。
C_c, B_b, S_t	row chunk、column band 和 AR 中的 stripe。
$h_c, h_s, w_b, w_b^{\max}$	chunk、stripe 行数及 band 宽度（含最大值）。
B	column bands 的数量。
$F, \mathcal{F}, \mathcal{T}$	frontier 深度、frontier 工作集和 tail 工作集。
L	RS 的 active-window depth。
R	可复用 stage slots 的数量；在 RS 中对应 reducer stream slots，在 AR 中直接决定 compact stripe staging 的容量。
Q_d	destination rank d 上的 symmetric staging workspace。
q, r	分别表示 RS window slot 和 AR compact stripe slot。

Q_d 混淆，本文正文将这两个位置记为 q_0, q_1 ； Q_d 始终只表示 destination rank d 上的完整 symmetric staging workspace。

若每个 band 的最大宽度为 $w_b^{\max}$ 、元素字节数为 e ，RS 的对称暂存分配规模可近似写为

$$M_{RS}^{\text{stage}} \approx L \cdot B \cdot p_\ell \cdot h_c \cdot w_b^{\max} \cdot e + M_{RS}^{\text{flag}}, \quad (2)$$

其中 B 为 band 数量， M_{RS}^{flag} 为 tickets 元数据。local-seed-direct 虽省去本地 contribution 的远端写入，当前 rank-indexed 布局仍为 p_ℓ 个来源位置预留行空间；

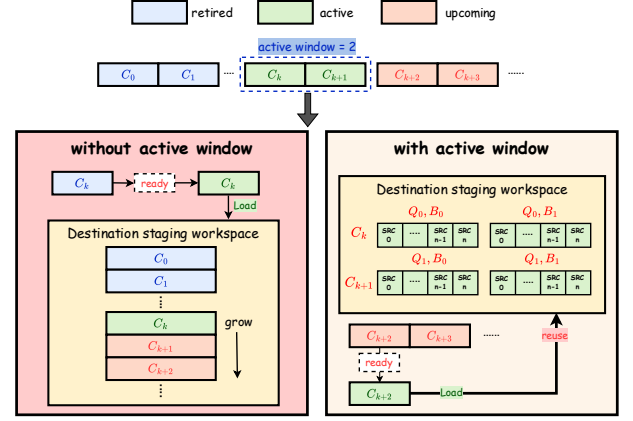


图 5. 活动窗口化对称暂存中的 slot 复用。

RS 的 R 只复用 reducer pipeline，远端驻留上界仍由 L 给出。图 3 给出相应的映射包络。

AR 将 physical staging 单元压缩为 stripe 后，其主要数据规模为

$$M_{AR}^{\text{stage}} \approx R \cdot p_\ell \cdot h_s \cdot w_b^{\max} \cdot e + M_{AR}^{\text{flag}}. \quad (3)$$

因此 R 在 AR 中直接控制 compact stripe staging 的容量。

4 算子实例化与运行时调优

本文在 Triton-distributed 的 Triton kernel 与 one-sided collective 基础上实例化该协议 [42]。AG-GEMM 区分本地立即可用与远端渐进可用输入；GEMM-RS 与 GEMM-AR 共享 panel-ready、frontier-first 和 ready/free 回收骨架，只在 physical staging 映射和输出语义上分开处理。伪代码描述数据流和状态约束，而不绑定具体 kernel API。

4.1 AG-GEMM 实例化 (AG-GEMM Instantiation)

在 AG-GEMM 中，AllGather 逐步形成 $\{A_0, \dots, A_{p-1}\}$ 的 activation 视图，rank s 用本地 W_s 计算

$$D_{s,u} = A_u W_s, \quad u \in [0, p), \quad (4)$$

其中 $D_{s,u}$ 占据 D_s 中与 source rank u 对应的不重叠行区间。 A_s 在调用开始时已经可见，故 $D_{s,s}$ 立即进入 local GEMM path。

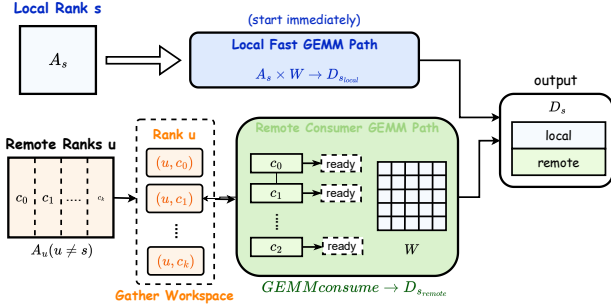


图 6. AG-GEMM 的 local/remote 双路径。

对 $u \neq s$, $A_u[C_c]$ 到达 gather workspace 后, copy stream 发布 row-tile barrier, 依赖该 chunk 的 remote GEMM tile 在设备侧等待 barrier 后读取输入。local 与 remote path 写入不重叠区域, 可并发推进; 细粒度路径仅用于 intra-node All2All、非 persistent consumer 且启用 row-tile barrier 的条件, 否则回退为 rank-ready, 数学输出不变。图 6 给出两条路径的依赖关系。

算法 1 给出该过程。copy stream 与 remote consumer kernel 并发启动; 每个输出 tile 在设备侧等待其依赖的 (u, c) barriers, 而非在 host 侧按 ready 事件逐个入队。等待边界因而下沉到实际读取的输入 chunks。

4.2 统一 RS/AR 实例化 (Unified RS/AR Instantiation)

4.2.1 共享的 Panel-Ready 与 Frontier-First 逻辑 (Shared Panel-Ready and Frontier-First Logic)

GEMM-RS 和 GEMM-AR 的 collective 终态不同, 但上游生产逻辑相同: source rank 先产生 panel 级 partial result, 再把相应 contribution 投递到 consumer 可见的位置。记 $P_{s \rightarrow d}[C_c, B_b]$ 为 source rank s 面向 destination rank d 生成的局部 contribution。GEMM-RS 在 destination d 上的 panel 输出满足

$$D_d[C_c, B_b] = \sum_{s=0}^{p-1} P_{s \rightarrow d}[C_c, B_b]. \quad (5)$$

GEMM-AR 则要求每个 rank 获得同一聚合值; 其逻辑 panel 仍由所有 ranks 的 partial results 归约而

算法 1: Consumer-anchored AG-GEMM

输入: 本地分片 A_s, W_s ; row chunks

$\{C_c\}_{c=0}^{C-1}$; 并行组大小 p

输出: 输出分片 D_s

- 1 初始化 gather workspace 与 $\text{ready}[u, c] \leftarrow 0$;
- 2 **foreach** $u \neq s$ 且 $c \in [0, C)$ **do**
- 3 异步传输 $A_u[C_c]$; 完成后发布
 $\text{ready}[u, c] \leftarrow 1$;
- 4 并发提交 local GEMM: $D_{s,s} \leftarrow A_s W_s$;
- 5 启动 remote GEMM consumer;
- 6 **foreach** 依赖远端输入的输出 tile $\mathcal{G}$ **do**
- 7 $\mathcal{D}(\mathcal{G}) \leftarrow$ 该 tile 读取的 (u, c) 集合;
- 8 **foreach** $(u, c) \in \mathcal{D}(\mathcal{G})$ **do**
- 9 设备侧等待 $\text{ready}[u, c] = 1$;
- 10 计算并写入 $\mathcal{G}$ 对应的 $D_{s,u}[C_c]$;
- 11 同步两条 consumer path, 并返回 D_s ;

成。两种情况下, consumer 都不必等待其他 chunks 或 bands, 只需等待当前 (c, b) 所需的 contributions。

运行时构造 $\mathcal{O} = [\mathcal{F}; \mathcal{T}]$, 先将 $\mathcal{F}$ 的 panels 送入 producer 队列, 再以 $\mathcal{T}$ 持续供给。RS 在 destination 的 (c, b) panel 完成后发布 panel-ready, source 等待该信号和前序 free ticket, 写入 contribution 并发布 arrival; AR 在 panel 完成后逐 stripe 发布 stripe-ready。两条路径的 consumer 都只等待当前单元所需的 ready 与 arrivals, 完成后发布 free。算法 2 分别给出 RS 的 $\text{chunk_signal}/(q, b)$ tickets 与 AR 的 $\text{stripe-ready}/r$ tickets, 二者共享依赖暴露和 frontier/tail 原则而非同一缓冲区粒度。

4.2.2 RS 特定的窗口暂存 (RS-Specific Windowed Staging)

RS 中, 逻辑 panel (c, b) 使用 $q = c \bmod L$ 与当前 ticket $\tau(c, b)$ 。启用 local-seed-direct 时, 本地 contribution 直接作为 reduction seed, 只有 $s \neq d$ 的远端 contribution 写入 $Q_d[q, b, s]$ 。source 等待前序 free ticket 和 $\text{chunk_signal}[d, c, b]$ 后发布 arrival; destination 等待本地 signal 与所需

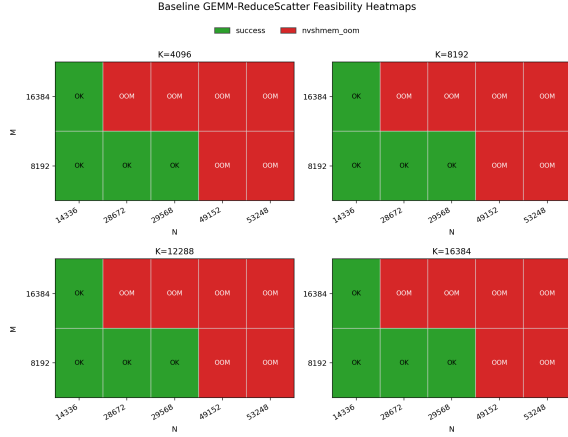


图 7. 基线 GEMM-RS 的 NVSHMEM 对称内存分配状态。

arrivals, 完成式 (5) 后写入 $\tau(c, b)$ 的 free flag。free 因而同时表示可复用性与该物理位置的循环轮次。

远端 panel 驻留由活动窗口而非总 chunk 数决定。 L 增大可保持更深的 in-flight 队列, 过小则可能等待 free; $j = (cB+b) \bmod R$ 仅复用 reducer stream, RS 的远端 staging envelope 仍由式 (2) 中的 L 、 B 、 p_ℓ 和 panel 形状决定。

图 7 汇总已存档 MN 与 K 条件上的基线 RS NVSHMEM 分配状态。运行时以式 (2) 及实际 h_c 、 w_b 、 B 和数据类型检查预算; 该约束在第 4.3 节作为 RS 候选的静态过滤条件。

4.2.3 AR 特定的 Stripe 暂存 (AR-Specific Stripe Staging). AR 仍按照 panel (c, b) 进行 frontier-first 生产, 以保留与 GEMM 上游输出相匹配的逻辑 ready 边界。为了避免把完整 panel 长时间固定在 symmetric memory 中, 运行时在物理层把 panel 切分为 stripes S_t 。令 $\mathcal{U}$ 表示按 $\mathcal{O}$ 依次展开 panels 内 stripes 得到的本轮任务序列, $\text{id}(c, b, t)$ 是 (c, b, t) 在 $\mathcal{U}$ 中从零开始的位置。每个 stripe 使用

$$r = \text{id}(c, b, t) \bmod R \quad (6)$$

选择 compact stage slot, 并获得当前 ticket 与该 slot 的前序 free ticket。source ranks 等待这两个条件中的前序 free ticket 与本地 stripe-ready ticket, 随后将 stripe contribution 写入相应 peer-visible slot 并发布 destination arrival ticket。consumer 等待本地

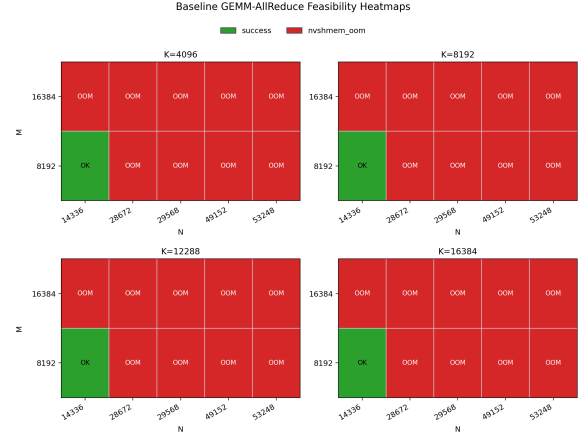


图 8. 基线 GEMM-AR 的 NVSHMEM 对称内存分配状态。

stripe-ready 与所有远端 arrival tickets 后执行

$$D[C_c, B_b, S_t] = \sum_{s=0}^{p-1} P_s[C_c, B_b, S_t], \quad (7)$$

归约完成后将当前 ticket 写入 free flag, 以释放 r 供后续 stripe 使用。

AR 的两层粒度由此具有不同职责: panel 控制生产顺序和 frontier-first 调度, stripe 控制 physical staging 和紧凑聚合。与 RS 不同, R 在 AR 中直接定义同时可驻留的 compact stripe slots, 因此是式 (3) 的核心参数。该分离既避免为完整 panel 预留过多对称内存, 也不改变 producer 按 panel 生成 partial result 的计算结构。

图 8 汇总相同已存档形状上的基线 AR NVSHMEM 分配状态。AR 的 stripe staging 容量受 R 与 h_s 控制, 静态检查以式 (3) 计入 data、flags、对齐和聚合元数据。

4.3 约束感知配置搜索 (Constraint-Aware Search)

参数彼此耦合: h_c 、 w_b 同时影响 ready 粒度、GEMM tile 与消息大小, F 决定 wake-up work 的提前程度, L 和 R 分别限制 RS window 与 AR stripe staging, lanes、consumer 并发度和 SM 配额决定资源竞争。 F 与对称内存约束均依赖 shape, 故孤立选择单一参数不能保证配置既可执行又高效。

算法 2: Frontier-first collective-GEMM runtime

输入: 模式 $o \in \{\text{RS}, \text{AR}\}$; panels (c, b) ; frontier 深度 F ; 窗口/slot 参数 L, R

输出: RS 的 shard 输出或 AR 的聚合输出

```

1  $\mathcal{O} \leftarrow [\mathcal{F}; \mathcal{T}]$ , 其中  $\mathcal{F}$  为前  $F$  个 chunks 的 wake-up panels;
2 if  $o = \text{RS}$  then
3   按  $\mathcal{O}$  启动 panelized GEMM producer; 完成 panel 后发布  $\text{ready}[d, c, b]$ ;
4   foreach 逻辑  $\text{panel}(c, b)$  和  $\text{destination } d$  do
5      $q \leftarrow c \bmod L, j \leftarrow (cB + b) \bmod R$ ;
6     等待前序  $\text{free}[q, b]$  与  $\text{ready}[d, c, b]$ ; 写入  $Q_d[q, b, s]$  并发布 arrival;
7     reducer  $j$  等待 local seed 与全部 arrivals, 计算  $D_d[C_c, B_b]$ , 再发布  $\text{free}[q, b]$ ;
8 else
9   按  $\mathcal{O}$  展开 stripe 任务序列  $\mathcal{U}$ , 为每个任务分配当前/前序 free tickets;
10  foreach  $(c, b, t) \in \mathcal{U}$  do
11     $r \leftarrow \text{id}(c, b, t) \bmod R$ ; 等待前序  $\text{free}[r]$  与  $\text{stripe\_ready}[c, b, t]$ ;
12    写入各 source contribution 并发布 arrivals; 等待全部 arrivals 后聚合该 stripe, 发布当前
    free[r];

```

给定算子类型 o 、问题形状 $x = (M, N, K, p)$ 和硬件环境 H , 本文把一个候选配置写为

$$\theta = (h_c, w_b, F, L, R, \ell, \rho, \chi), \quad (8)$$

其中 ℓ 为通信 lane/并发度, ρ 为 consumer/reducer 资源配额, χ 为算子特有离散选项。搜索空间按算子语义生成: AG 不使用 RS 的 L 或 AR 的 R , RS 的 R 复用 reducer pipeline, AR 的 R 表示 stripe staging 容量。

候选配置在被执行前需要通过静态可行性检查。检查内容包括 kernel tile 与矩阵形状是否匹配、frontier 和窗口是否超过逻辑工作数、并发度是否满足运行时限制, 以及 symmetric memory 是否满足预算 $M_{\text{sym}}^{\text{budget}}$ 。对 RS 和 AR, 内存约束分别写为

$$M_o^{\text{stage}}(\theta) + M_o^{\text{meta}}(\theta) \leq M_{\text{sym}}^{\text{budget}}, \quad o \in \{\text{RS}, \text{AR}\}, \quad (9)$$

其中 M_o^{stage} 由式 (2) 或式 (3) 确定, M_o^{meta} 覆盖 flags、对齐和运行时元数据。该筛选将 allocation failure 与明显不合法的调度组合排除在短测量之外。

在可行集合上, 搜索按 (h_c, L, B) 形成结构 bucket, 以实现启发式保留少量 stage-slot、lane

算法 3: Constraint-aware two-stage search

输入: 算子 o 、形状 $x = (M, N, K, p)$ 、硬件描述 H 、候选空间 Θ_o

输出: 配置 θ^*

```

1 if  $\text{cache}(o, x, H)$  存在 then
2   return  $\text{cache}(o, x, H)$ ;
3  $\mathcal{C} \leftarrow \{\theta \in \Theta_o : \text{Feasible}(\theta, x, H)\}$ ;
4  $\mathcal{B} \leftarrow \text{Bucket}(\mathcal{C}; h_c, L, B)$ ;
5  $\mathcal{S}_1 \leftarrow \text{Shortlist}(\mathcal{B}, k)$ ; 短测量  $\tilde{T}(\theta)$ ;
6  $\mathcal{S}_2 \leftarrow \text{Refine}(\mathcal{S}_1, r)$ ; 短测量  $\tilde{T}(\theta)$ ;
7  $\mathcal{V} \leftarrow \text{Validate}(\text{Top}_v(\mathcal{S}_1 \cup \mathcal{S}_2))$ ;
8  $\theta^* \leftarrow \arg \min_{\theta \in \mathcal{V}} T(\theta)$ ; 缓存并返回  $\theta^*$ ;

```

和 frontier 变体并短测量; 第一阶段的低延迟结构候选进入第二阶段, 对 steady/tail SM 配额做聚焦扩展, 前列候选再做较长验证并按 (o, x, H) 缓存。搜索只为既定的 ready/arrival/free 协议选择资源包络与推进深度, 不改变数据依赖或声明全局最优。

搜索先排除不满足对称内存和运行时约束的候选，再在有效集合中比较重叠收益；其实际测量成本与结果在实验章节报告。

5 实验评估

本章从算子级、层级集成和完整模型推理三个层次评估本文 collective-GEMM 计算通信重叠框架，并通过机制消融解释性能收益的来源。实验按照从总体收益到运行时机制的顺序组织：先说明测试平台、工作负载、基线和计时口径，再给出 AG-GEMM、GEMM-RS 与 GEMM-AR 的算子级结果，随后考察 TP Attention、TP MLP 和完整模型 prefill，最后分析 ready 粒度、frontier-first 调度、活动窗口暂存以及约束感知搜索。

本章以 A100 结果作为主要性能证据，以海光 K100_AI 结果作为第二平台上的实现和机制可用性证据。两个平台的绝对时延不直接横向比较；每个平台内的 speedup 均相对于对应平台上的匹配基线归一化。对于作者侧原始记录以最低值和最高值保存的算子结果，图中柱体取该观测区间的中点、误差线表示观测范围；它们不被解释为置信区间或未记录重复次数的统计估计。

5.1 实验设置

5.1.1 测试平台与软件环境. 历史主性能结果来自单机 4 张 A100-80GB GPU，第二平台为单机 4 张海光 K100_AI-64GB 加速卡。A100 覆盖三类算子、层级集成和主要机制消融，K100_AI 提供算子级和部分端到端 prefill 结果。受控 A100 执行环境为：4 × NVIDIA A100-SXM4-80GB，任意 GPU 对之间为 NVLink NV12 互连；双路 AMD EPYC 7742 主机、约 1 TiB DRAM、Ubuntu 24.04 (Linux 5.15)、NVIDIA Driver 580.65.06；软件栈为 Python 3.12、PyTorch 2.7.0 (CUDA 12.6)、NCCL 2.26.2、Triton 3.4.0 与 NVSHMEM 3.3.9。K100_AI 侧软件栈实测为 DTK 26.04、Python 3.11、PyTorch 2.7.1 (DTK 构建, HIP 6.3) 与 Triton 3.1.0；基线 collective 经 RCCL 2.22.3 完成，而本文 one-sided 对称暂存与同步原语由 DUSHMEM 3.2.5 提供——它是 A100 侧

表 3. 端到端工作负载与计时口径

工作负载	模型与输入	精度与计时
TP Attention	Llama3-70B / Qwen2-72B; $B=64$, $S=128$, $M=8192$	BF16; warmup 20 / iter 100; 仅计 CUDA Graph replay
TP MLP	同上模型; (M, N, K) 见正文	BF16; warmup 20 / iter 100; 仅计 CUDA Graph replay
Full E2E prefill	完整 TP-sharded Transformer body, 不含 LM head	BF16; warmup 10 / iter 20; CUDA events 仅测 replay

NVSHMEM 所代表的 symmetric-memory 与 one-sided 通信思想向 DCU 平台的对应迁移实现，因此同一套 ready/ticket/active-window 协议无需修改即可在该平台实例化。上述配置用于固定后续复现条件；历史 A100 与 K100_AI 图表仍按其原始平台内的归一化 speedup 解释，本文不追溯性地补写其缺失的 driver、拓扑或逐 rank 时延。

5.1.2 工作负载、基线与计时口径. 算子级实验覆盖 AG-GEMM、GEMM-RS 和 GEMM-AR。端到端实验分别覆盖 TP Attention、TP MLP 和完整模型推理；模型为 Llama3-70B 和 Qwen2-72B，运行阶段为 prefill。E2E 使用 4 路张量并行、BF16、batch size 64 和 sequence length 128，因而 prefill 的 token-row 维度为 $M = 64 \times 128 = 8192$ 。MLP 投影对应的 (M, N, K) 分别为 Llama3-70B 的 (8192, 28672, 8192) 与 Qwen2-72B 的 (8192, 29568, 8192)。

表 3 固定端到端面板的工作负载与计时口径。Attention 和 MLP 的单次 launch 分别先以 20 次 warmup 预热，再对 100 次 replay 取平均；完整模型 prefill 使用 10 次 warmup 和 20 次 replay。计时仅覆盖已选路径的 replay：路径前后执行 CUDA synchronize，Torch 与 Triton 段之间执行 NVSHMEM barrier，编译与 graph capture 不计入。表中的历史面板保存为单次 launch 的最终 speedup，不以此构造误差线、置信区间或 rank-max latency。

算子级主对比沿用 no-overlap 归一化约定：以串行无重叠路径为基准置 1。该路径在 A100 平台由 PyTorch/NCCL、在海光 K100_AI 平台由 PyTorch/RCCL 组成；对三条链路，基准均按相应

collective 与 GEMM 的串行组合实现。算子级图中除 no-overlap 与本文路径外，还按算子在可运行处展示 FLUX 与 FlashOverlap 等对比方法（AG 面板含 FLUX 与 Triton-distributed，RS 面板含 FLUX 与 FlashOverlap，AR 面板含 FlashOverlap）；各方法以该基准归一化，回答重叠相对无重叠的收益。机制消融中的 AG ready 粒度和 RS frontier 调度均以 Triton-distributed 参考路径为基准。frontier 比较报告 uniform schedule 与 frontier-first schedule 两个实现变体之间的差异，不把它写作只改变单一运行时策略的因果消融；两套口径不放入同一张图。

端到端图报告各条 AG/RS 路径相对于语义等价 Torch no-overlap 参考路径的 speedup：

$$\text{Speedup}_{\text{torch}} = \frac{T_{\text{torch}}}{T_{\text{variant}}}. \quad (10)$$

所有路径均先完成 BF16 correctness check，再进入表 3 所列的性能循环。Attention、MLP 和完整模型均使用 $(\text{atol}, \text{rtol}) = (2 \times 10^{-2}, 2 \times 10^{-2})$ ；实现将输出（完整模型为 softmax logits）与 Torch/HuggingFace golden 比对，通过 all close 后才执行计时。三条算子链的逐链协议、阈值与通过记录见表 4。

计时由 CUDA events 完成：warmup 结束后记录起始 event，连续执行指定次数，在当前 stream 同步后以总 event 时间除以 iterations。图 12 的每个条件保留一条历史 launch speedup，故不报告误差线，也不将单条记录解释为统计估计。历史摘要保存的是最终 speedup 而非完整逐 rank timing，本文不将其标注为 rank-max step latency。

5.1.3 正确性与槽位复用验证. 所有链路在每个 launch 进入计时循环前先执行逐 rank 数值检查：实现输出与语义等价的 Torch reference 以 all close 断言比较，任一 rank 失败即中止该 launch；检查同时拒绝在均值过小的近零张量上产生空真通过。各链的 reference 与阈值见表 4。方法节的三条 ticket 不变量——数据写入完成后才发布 arrival、consumer 收齐当前依赖 ticket 前不读取、free(t) 发布后才允许覆盖同一 physical slot——由该数值检查与全部 launch 在外层时限内无挂起完成共同覆盖。

阈值由 BF16 的数值结构决定：其 8 位有效尾数使相邻可表示数的相对间隔约为 $2^{-7} \approx 0.78\%$ 。参考实现与本文路径在 FP32 累加后经不同归约顺序与分块再舍入回 BF16，仅舍入顺序差异即造成 1-2 个 ulp（约 0.8%–1.6% 相对偏差）。据此，算子链阈值 2×10^{-2} 约合 2.5 个 ulp，覆盖合法重排偏差并保留判定余量；端到端比较跨越约 80 层 Transformer，偏差逐层复合，同一量级与其模型级测试的常见口径一致。AG-GEMM 的 golden 比较采用 10^{-3} ，低于单个 ulp，要求两者几乎逐位一致，进一步排除阈值过松掩盖偏差的情形。

槽位复用压力由活动窗口 sweep 显式构造：该批每段含 8 个逻辑 chunks，而 $L \in \{1, 2, 4\}$ 且 staging slots 为 4，逻辑工作多于驻留槽数，同一 physical slot 被多轮覆盖后仍须数值一致。AG 侧的 ready 粒度由 2048 行降至 256 行，形成多次依赖发布与消费。死锁与超时按外层时限判定（算子级 240 s、端到端 1800 s）：超时或任一 rank 比较失败记为该 launch 失败，重试前的失败尝试不计入通过数。表 4 汇总各链的协议与通过记录；GEMM-AR 与 RS 采用同一协议，并在 K100_AI 第二平台完成逐 rank 验证。

5.2 算子级性能

算子级实验回答统一 producer-consumer 重叠机制是否能够在三类 collective-GEMM 链上转化为可观测的时延收益。图 9 给出 A100 主平台结果，图 10 给出海光 K100_AI 平台结果。两幅图分别在各自平台内归一化，不能据此比较两个平台的绝对执行速度。

A100 结果承担主要性能结论，DCU 结果作为第二平台上的实现和机制可用性证据。对于 Triton-distributed 无法完成对称内存分配的 shape，结果图保留日志中的失败类别，因为这类失败反映了 staging footprint 对系统可行性的影响。

5.2.1 可行性感知对比. 仅比较成功运行的 shape 会掩盖对称暂存设计的系统影响。图 11 在 GEMM-RS 和 GEMM-AR 上同时保留可运行结果与失败结果，说明 bounded staging 不只是 kernel 调度优化，也决定了某些问题形状能否进入测量阶段。图中的 SYM-MEM 表示原始 Triton-distributed 路径在

表 4. 计时前正确性检查协议与通过记录。通过计数包含开发期记录：早期批次的失败不进入任何性能图，也不表示最终版本的失败率。

链路	Reference (逐 rank)	BF16 阈值	复用压力与通过记录
AG-GEMM	Torch all-gather+GEMM golden	10^{-3}	实验 121/129 launch 通过，8 次失败集中于同一 shape 的两个早期调试批次。
GEMM-RS	Torch GEMM+reduce-scatter	2×10^{-2}	强制 slot 绕回复用；活动窗口 60/60，受控 order 批 11/12（1 次为计时阶段超时，其数值检查已通过）。
GEMM-AR	Torch GEMM+all-reduce	2×10^{-2}	强制 slot 绕回复用；与 RS 同协议，所有实验均已通过。

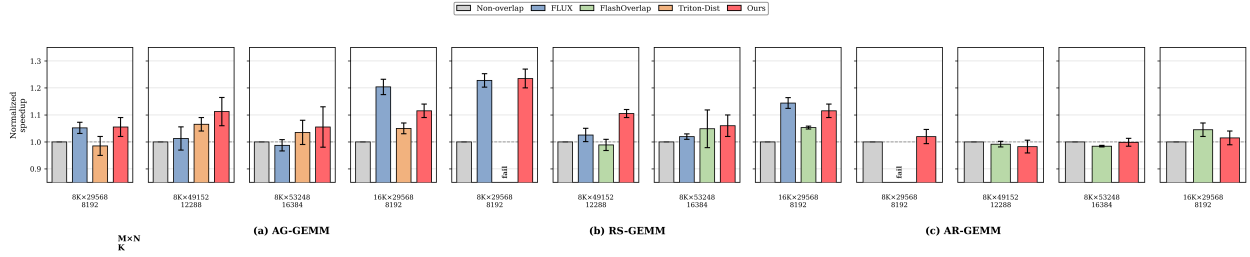


图 9. A100 平台上三类 collective-GEMM 链的算子级归一化性能。

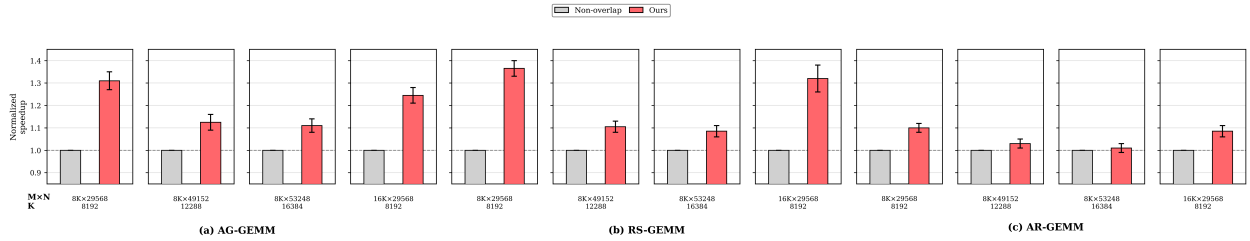


图 10. 海光 K100_AI 平台上三类 collective-GEMM 链的算子级归一化性能。

日志中记录的 symmetric-memory allocation failure; UNSUPPORTED 表示基线不支持该条件，FAILED 表示日志未进一步归因的一般运行失败。三者均不被写作 OOM。

该结果与第 3.3 节的设计分析相互对应：若每个逻辑单元永久绑定一块对称暂存位置，分配需求会随逻辑工作集增长；活动窗口通过复用有限 physical slots 将 resident footprint 限制在可控范围内。

5.3 端到端 Prefill 评估

算子级实验说明单个 collective-GEMM 链能够受益于细粒度重叠，但模型执行还包含投影、注意力、激活函数、归一化以及其他无法被本文路径替代的工作。端到端评估用于回答算子级收益能否传递到 TP Attention、TP MLP 和完整模型推理。当前结果集中在

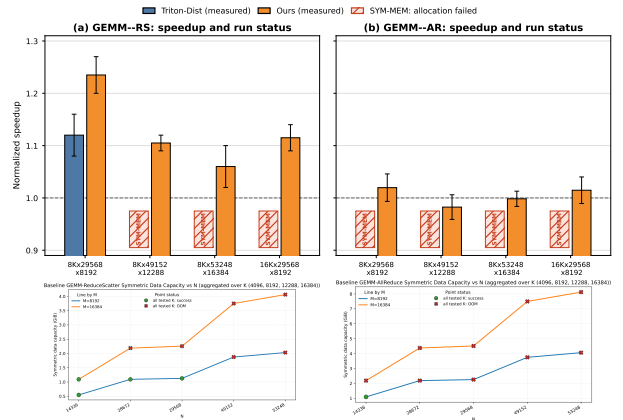


图 11. A100 平台上 GEMM-RS/AR 的可行性感知算子对比；斜线状态表示日志记录的对称内存分配失败。

prefill, 因为该阶段的 token 数量使 AG/RS 链具有更明显的重叠空间。

5.3.1 TP Attention、TP MLP 与 Full E2E Prefill. 图 12 汇总三个工作负载的 prefill speedup。每个模型、平台和路径只对应一条已整理的历史记录, 因而没有添加误差棒。A100 面板同时给出 Torch no-overlap、Triton-distributed、Baseline AG/Proposed RS、Proposed AG/Baseline RS 与 Proposed AG/Proposed RS。组合路径在六个 A100 workload-model 记录中均为图内最高值: Attention 的 Llama3-70B 与 Qwen2-72B 分别为 $1.20\times$ 与 $1.23\times$, MLP 为 $1.13\times$ 与 $1.11\times$, 完整 E2E 为 $1.05\times$ 与 $1.09\times$ 。两条单侧替换路径与 Triton-distributed 的记录显示, AG 与 RS 在不同工作负载中的贡献并不相同; 组合路径的数值在所记录条件下反映两条路径共同启用后的结果。

K100_AI 当前只保留 Torch no-overlap 与 Proposed AG/Proposed RS 两条路径。Attention、MLP 和 Full E2E 的组合路径 speedup 分别落在 $1.28\sim 1.30\times$ 、 $1.22\sim 1.23\times$ 和 $1.20\sim 1.22\times$ 范围内。第二平台上的记录同样表明组合路径能够相对于 Torch 基线获得 prefill 收益; 由于其未记录 Triton-distributed 与两条单侧替换路径, 图中不以空柱补齐这些条件。两个平台只在各自执行栈和所测模型条件内解释, 不用于跨平台绝对时延排名。

这些 E2E 记录将 AG/RS 的单侧替换和组合路径并列展示, 但每个条件仍只有一条历史记录。后续若在冻结的环境、模型、batch、sequence length 与计时协议下取得独立重复 launch, 才能将该图扩展为带离散度估计的受控 E2E 消融。

5.4 机制消融

端到端 speedup 只能说明组合路径的收益, 不能说明收益来自哪一项机制。本节按照第 3 章的设计顺序, 分析 ready 粒度、frontier-first 调度和活动窗口暂存。每组比较均明确对照范围; frontier 数据来自 uniform schedule 与 frontier-first schedule 的实现变体比较, 不作为只改变单一运行时策略的因果消融。

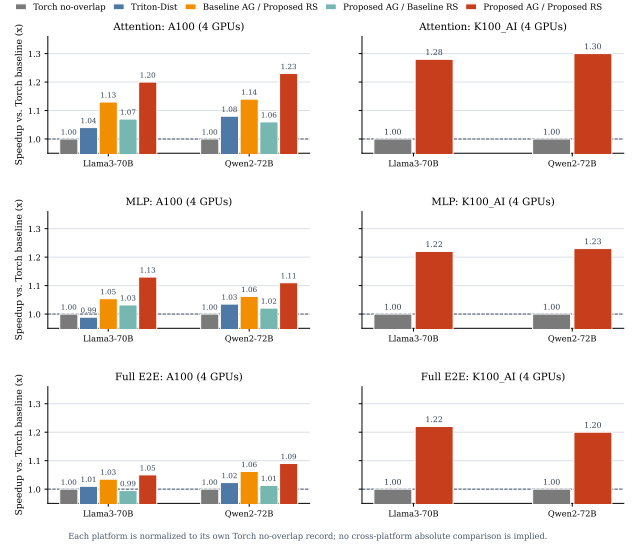


图 12. TP Attention、TP MLP 与完整模型的 prefill speedup; 各平台均相对其 Torch no-overlap 基准归一化。

5.4.1 细粒度 Ready 单元. AG ready 粒度实验比较 rank-ready、固定 row-tile-ready 粒度和启发式 tile-ready。rank-ready 只有在更大的远端数据区域完成后才发布可消费状态; tile-ready 将同一远端 activation shard 划分为多个可独立驱动 GEMM 的 row chunks。本消融以 rank-max 总 latency 及相对 Triton-distributed 参考路径的 speedup 比较不同 ready 粒度。图 13 基于三个代表性矩阵形状展示该消融。本节各消融的形状均为 case-study 选择: RS frontier 与活动窗口使用的 (8192, 29568, 8192) 对应 Qwen2-72B TP-prefill 的 MLP 投影, (8192, 49152, 12288) 与 (8192, 53248, 16384) 为其加宽、加深的压力变体; AG ready 粒度的三例取自自己测集中通信/计算比例不同的形状。它们不代表全部已测条件, 更大范围的算子级汇总见图 9。

如果 tile 切分过粗, ready 事件仍不能显著提前; 如果切分过细, 额外的 signal、polling 和任务调度开销可能抵消重叠收益。因此, 最优点不是最小 tile 的简单选择, 而是由 GEMM 消费边界和通信消息大小共同决定的折中区域。

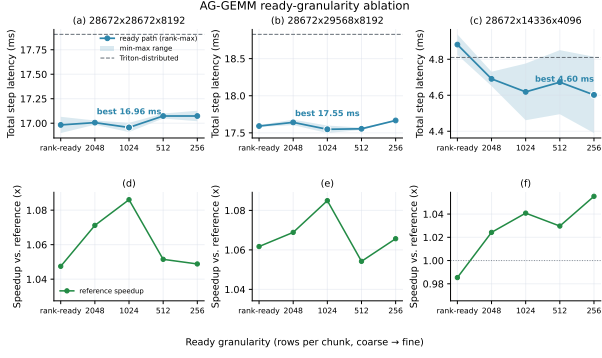


图 13. AG-GEMM 的 ready 粒度消融; 上行为 rank-max latency, 下行为相对参考路径的 speedup。

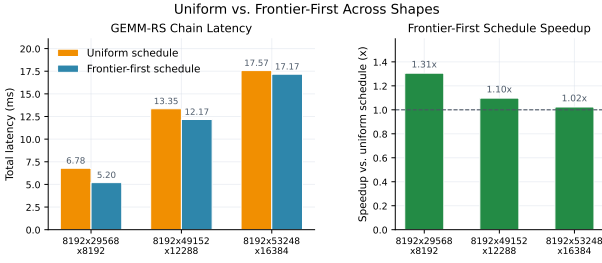


图 14. 常规 uniform 启动与本文 consumer-anchored frontier-first 启动下的 GEMM-RS 链路时延。

5.4.2 Consumer-Anchored Frontier 调度.

细粒度 ready 暴露可消费边界后, 常规 uniform schedule 仍沿 producer 的既定顺序推进。本文的 consumer-anchored frontier-first 策略优先发布位于当前 consumer wake-up frontier 的 panels, 并以 tail work 维持后续供给。图 14 在匹配的外部 shape 和运行时资源设置下, 将该提出路径与 uniform 基线置于完整 GEMM-RS 链路中比较。

图中所列配置表明, consumer-anchored frontier-first 能将 producer 的早期工作投入到缩短 consumer-critical path 的 panels, 从而在完整 GEMM-RS 路径上获得更短的链路时延; 其中一个 shape 的收益约为 1.02x, 应读作有限收益的边界情形而非统一加速。该图报告的是完整实现路径的效果, 不将全部差异归因于唯一的 panel 生产顺序。当前定量数据主要覆盖 GEMM-RS; GEMM-AR 共享 panel-ready 和 frontier-first 原则, 但其独立定量消融需要单独报告。

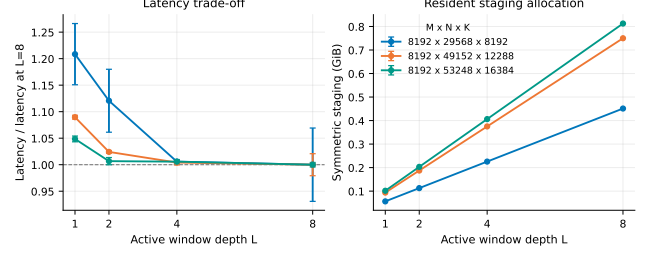


图 15. 活动窗口深度的 latency-symmetric-staging footprint 折中。

5.4.3 活动窗口化对称暂存. 活动窗口消融考察 resident symmetric staging footprint 与执行 latency 的关系。三个矩阵 shape 均使用 8 个逻辑 chunks, 并对 $L \in \{1, 2, 4, 8\}$ 各执行 5 次独立 launch; 60/60 个 launch 完成。以 $L = 8$ 为参考, $L = 1, 2, 4$ 的 latency ratio (中位数) 分别为 (1.2085, 1.1206, 1.0058)、(1.0896, 1.0242, 1.0041) 和 (1.0486, 1.0067, 1.0057), 对应的 symmetric staging footprint ratio 为 (0.125, 0.25, 0.5)。因此 $L = 4$ 在三个 shape 上都保留约一半的驻留 footprint, 同时将 latency 控制在 $L = 8$ 的约 0.4%–0.6% 以内; 该 footprint ratio 只针对 symmetric staging, 不表示总 GPU memory 减半。

较大的窗口允许更多 panel 同时处于 in-flight 状态, 可能降低 producer 等待, 但会扩大对称内存分配; 较小的窗口收紧资源包络, 却可能增加 slot reuse 带来的等待。因此, 窗口深度需要与问题形状和其他重叠参数联合选择。

5.5 约束感知配置搜索

不同配置会在 ready 粒度、生产顺序、通信并发度和对称暂存 footprint 之间形成耦合。本文采用约束感知的两阶段搜索: 先依据 shape、staging 预算和运行时限制筛选不可行组合, 再对少量代表性配置执行短测量。表 5 保留 exhaustive 与 two-stage 两种搜索方式的 launch 数和完成状态。

最新 A100 搜索记录覆盖 64 和 128 个原始候选配置。64-space 中, exhaustive 使用 $64 + 3 = 67$ 次逻辑测量, two-stage 使用 $4 + 6 + 3 = 13$ 次; 两者最终选择的配置仅在 *tail_sms* 上不同 (12 与 16), 且来自独

表 5. 约束感知 GEMM-RS 配置搜索的成本与最终验证结果

Shape	Space	Method	Logical measurements	Final latency (ms)	Status	Selected Config	Outcome
8192×29568×8192	64	Exhaustive	67	5.49	Completed	(512,4,4,6,12,2,2,1)	Full-space enumeration + final verification
8192×29568×8192	64	Two-stage	13	5.46	Completed	(512,4,4,6,16,2,2,1)	Two-stage selection + final verification
8192×29568×8192	128	Exhaustive	131	5.39	Completed	(512,4,4,6,24,2,2,1)	Full-space enumeration + final verification
8192×29568×8192	128	Two-stage	15	5.47	Completed	(512,4,4,6,12,2,2,1)	Two-stage selection + final verification

注：Selected Config 按 (*chunk_rows*, *active_chunk_window*, *stage_slots*, *steady_sms*, *tail_sms*, *comm_lanes*, *n_bands*, *frontier_chunks*) 给出。Space 为 exhaustive 枚举的原始参数空间大小；two-stage 在同一原始参数域内先筛选结构配置，再展开 SM 预算组合。Logical measurements 包含快速搜索与最终 top-3 verification；失败后的重试保留在执行日志中，不额外计入该逻辑预算。Final latency 是最终 verification 中的 rank-max 链路总时延。

立的最终验证，分别记录为 5.49 和 5.46 ms。因而这 0.03 ms (约 0.55%) 的数值反转不应解释为 two-stage 优于 exhaustive，亦不表示其发现了枚举域外的候选。128-space 中，exhaustive 的 $128 + 3 = 131$ 次逻辑测量得到 5.39 ms，而 two-stage 的 $4 + 8 + 3 = 15$ 次逻辑测量得到 5.47 ms，距前者约 1.48%。两阶段搜索的主张是以显著更低的测量预算获得经最终验证、接近 exhaustive 的可执行配置，而非在所有形状和噪声条件下保证全空间最优或复现 exhaustive 的严格数值排序。

5.6 资源可行性与跨平台结果

图 11 与活动窗口实验共同表明，运行时在满足 symmetric-memory 预算的前提下选择可维持有效重叠的 resident envelope；其中 SYM-MEM 始终表示 NVSHMEM symmetric-memory allocation failure，而非 speedup 为零或普通 CUDA OOM。K100_AI 的算子级和 prefill 结果仅作为第二平台的实现与机制可用性证据。由于平台的硬件、驱动栈和通信运行时不同，本文只在各平台内部解释 normalized speedup，不作跨平台绝对 latency 或多节点可扩展性推断。

6 结论

本文将张量并行 collective-GEMM 链表述为 producer-consumer 依赖的及时解除：运行时以 consumer 的最小可启动依赖定义 ready，以 frontier/tail 组织 producer 推进，并以活动窗口限制 one-sided symmetric staging 的驻留。AG 的 row chunk、RS 的

chunk-band panel 和 AR 的 stripe 是该协议的不同实例；RS/AR 的 ready/free tickets 使物理 slot 只在 consumer 完成后复用。形状、窗口和对称内存预算在测量前进入可行性筛选。

现有证据覆盖 A100 与 K100_AI 上的三类算子、AG/RS 张量并行 prefill 以及 ready 粒度、frontier 实现变体、活动窗口和配置搜索。算子结果显示细粒度依赖暴露带来的重叠机会；所测 Attention、MLP 与完整模型 prefill 记录显示该收益可传递到 AG/RS 路径。活动窗口量化了 symmetric staging 驻留与流水线开销的折中，两阶段流程在有限预算下完成候选选择，但不承诺全空间最优。

结论受当前实现和评价范围约束：端到端结果只覆盖组合后的 AG/RS 张量并行 prefill，GEMM-AR 仍限于算子级；单机四卡的平台内归一化结果不能推出跨平台绝对时延或多节点可扩展性。不同系统、模型形状和拓扑仍需重新选择 frontier 深度、ready 粒度与暂存窗口。后续工作将补充分离 AG/RS 贡献的完整模型消融、AR 端到端集成，以及更多拓扑、工作负载和可复现基线条件下的配置选择评估。

References

- [1] Anirudha Agrawal, Shaizeen Aga, Suchita Pati, and Mahzabeen Islam. 2025. ConCCL: Optimizing ML Concurrent Computation and Communication with GPU DMA Engines. In *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 1–11. doi:10.1109/ISPASS64960.2025.00018
- [2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).

- [3] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, Haibin Lin, Xin Jin, and Xin Liu. 2024. FLUX: Fast Software-Based Communication Overlap on GPUs Through Kernel Fusion. *arXiv preprint arXiv:2406.06858* (2024).
- [4] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 3. 178–191. doi:10.1145/3620666.3651379
- [5] Meghan Cowan, Saeed Maleki, Madanlal Musuvathi, Olli Saarikivi, and Yifan Xiong. 2023. MSCCLang: Microsoft Collective Communication Language. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 502–514. doi:10.1145/3575693.3575724
- [6] DeepSeek-AI et al. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. *arXiv preprint arXiv:2606.19348* (2026).
- [7] Jiarui Fang and Shangchun Zhao. 2024. USP: A Unified Sequence Parallelism Approach for Long Context Generative AI. *arXiv preprint arXiv:2405.07719* (2024).
- [8] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. 2026. TokenWeave: Efficient Compute-Communication Overlap for Distributed LLM Inference. In *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*. arXiv:2505.11329 <https://openreview.net/forum?id=rh2Ylffkq6>
- [9] Ke Hong, Guohao Dai, Jiaming Xu, Qiuli Mao, Xiuhong Li, Jun Liu, Kangdi Chen, Yuhang Dong, and Yu Wang. 2024. FlashDecoding++: Faster Large Language Model Inference with Asynchronization, Flat GEMM Optimization, and Heuristics. In *Proceedings of the 4th Conference on Machine Learning and Systems (MLSys)*. https://proceedings.mlsys.org/paper_files/paper/2024/hash/5321b1dabdc2be188d796c21b733e8c7-Abstract-Conference.html
- [10] Ke Hong, Xiuhong Li, Minxu Liu, Qiuli Mao, Tianqi Wu, Zixiao Huang, Lufang Chen, Zhong Wang, Yichong Zhang, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2026. Efficient and Adaptable Overlapping for Computation and Communication via Signaling and Reordering. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. 1894–1911. doi:10.1145/3767295.3769370
- [11] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefer. 2025. Demystifying NCCL: An In-Depth Analysis of GPU Communication Protocols and Algorithms. In *Proceedings of the 32nd Annual Symposium on High-Performance Interconnects (HOTI)*. 48–59. doi:10.1109/HOTI66940.2025.00024
- [12] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the Computation and Communication Abstraction Barrier in Distributed Machine Learning Workloads. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 402–416. doi:10.1145/3503222.3507778
- [13] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 745–760. <https://www.usenix.org/conference/nsdi24/presentation/jiang-ziheng>
- [14] Rezaul Karim, Austin Wen, Wang Zongzuo, Weiwei Zhang, Yang Liu, and Walid Ahmed. 2026. CommFuse: Hiding Tail Latency via Communication Decomposition and Fusion for Distributed LLM Training. *arXiv preprint arXiv:2604.24013* (2026).
- [15] Kimi Team et al. 2026. Kimi K3: Open Frontier Intelligence. *arXiv preprint arXiv:2607.24653* (2026). <https://arxiv.org/abs/2607.24653>
- [16] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2022. Reducing Activation Recomputation in Large Transformer Models. *arXiv preprint arXiv:2205.05198* (2022).
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 611–626. doi:10.1145/3600006.3613165
- [18] Seonho Lee, Jihwan Oh, Seokjin Go, and Divya Mahajan. 2025. Characterizing Compute-Communication Overlap in GPU-Accelerated Distributed Deep Learning: Performance and Power Implications. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 353–355. doi:10.1109/ISPASS64960.2025.00041
- [19] Shengwei Li, Zhiqian Lai, Dongsheng Li, Yanqi Hao, Weijie Liu, Keshi Ge, Xiaoge Deng, and Kai Lu. 2025. Oases: Efficient Large-Scale Model Training on Commodity Servers via Overlapped and Automated Tensor Model Parallelism. *IEEE Transactions on Parallel and Distributed Systems (TPDS)* 36, 9 (2025), 1828–1840. doi:10.1109/TPDS.2025.3583165
- [20] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=WsRHpHH4s0>
- [21] Yijun Ma, Siyuan Shen, Tiancheng Chen, Akhil Langer, Jiri Kraus, Benjamin Glick, Craig Belusar, Jeff R. Hammond, and Torsten Hoefer. 2026. Demystifying NVSHMEM: A System-Level Analysis on Symmetric Memory and Device-Initiated Operations in GPU Communication. *arXiv preprint*

- arXiv:2606.05951* (2026).
- [22] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th Symposium on Operating Systems Principles (SOSP)*. 1–15. arXiv:1806.03377 doi:10.1145/3341301.3359646
- [23] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*. 1–15. doi:10.1145/3458817.3476209
- [24] NVIDIA. 2024. cuBLASmp: Parallel GEMM and Communication. <https://docs.nvidia.com/cuda/cublasmp/>.
- [25] NVIDIA. 2024. NCCL: NVIDIA Collective Communication Library. <https://github.com/NVIDIA/nccl>.
- [26] NVIDIA. 2024. NVSHMEM: OpenSHMEM-based PGAS for GPU. <https://developer.nvidia.com/nvshmem>.
- [27] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Ínigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 118–132. doi:10.1109/ISCA5907.7.2024.00019
- [28] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 2. 1146–1164. doi:10.1145/3620665.3640410
- [29] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1133–1150. doi:10.1145/3669940.3707256
- [30] PyTorch Foundation and TorchTitan Team. 2024. Async Tensor Parallelism (Async-TP). <https://discuss.pytorch.org/t/distributed-w-torchtitan-introducing-async-tensor-parallelism-in-pytorch/209487>.
- [31] Xinwei Qiang, Yue Guan, Zhengding Hu, Keren Zhou, Yufei Ding, and Adnan Aziz. 2026. Syncopate: Efficient Multi-GPU AI Kernels via Automatic Chunk-Centric Compute-Communication Overlap. In *Proceedings of the 20th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. arXiv:2601.20595
- [32] Qwen Team. 2026. Qwen3.8-Max: A New Bar for Coding and Cowork. <https://qwen.ai/blog?id=qwen3.8>
- [33] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 593–612. <https://www.usenix.org/conference/nsdi23/presentation/shah>
- [34] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [35] Weigao Sun, Zhen Qin, Weixuan Sun, Shidi Li, Dong Li, Xuyang Shen, Yu Qiao, and Yiran Zhong. 2024. CO2: Efficient Distributed Training with Full Communication-Computation Overlap. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=ZO5cn4IfaN>
- [36] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Jorgen Thelin, Nikhil Devanur, and Ion Stoica. 2020. Blink: Fast and Generic Collectives for Distributed ML. In *Proceedings of the 3rd Conference on Machine Learning and Systems (MLSys)*. https://proceedings.mlsys.org/paper_files/paper/2020/hash/cd3a9a55f7f3723133fa4a13628cdf03-Abstract.html
- [37] Guanhua Wang, Chengming Zhang, Zheyu Shen, Ang Li, and Olatunji Ruwase. 2024. Domino: Eliminating Communication in LLM Training via Generic Tensor Slicing and Overlapping. *arXiv preprint arXiv:2409.15241* (2024).
- [38] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyli, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, Sameer Kumar, Tongfei Guo, Yuanzhong Xu, and Zongwei Zhou. 2023. Overlap Communication with Dependent Computation via Decomposition in Large Deep Learning Models. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Vol. 1. 93–106. doi:10.1145/3567955.3567959
- [39] Bin Xiao and Lei Su. 2024. ISO: Overlap of Computation and Communication within Sequence For LLM Inference. *arXiv preprint arXiv:2409.11155* (2024).
- [40] Guanbin Xu, Zhihao Le, Yinhe Chen, Zhiqi Lin, Zewen Jin, Youshan Miao, and Cheng Li. 2025. AutoCCL: Automated Collective Communication Tuning for Accelerating Distributed and Parallel DNN Training. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 667–683. <https://www.usenix.org/conference/nsdi25/presentation/xu-guanbin>
- [41] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, Quan Chen, and Xin Liu. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. In *Proceedings of the 8th Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=fGgQS5VW09>
- [42] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun

Liang, Jidong Zhai, and Xin Liu. 2025. Triton-distributed: Programming Overlapping Kernels on Distributed AI Systems with the Triton Compiler. *arXiv preprint arXiv:2504.19442* (2025).

[43] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, and Xin Liu. 2025. TileLink: Generating Efficient Compute-Communication Overlapping Kernels using Tile-Centric Primitives. In *Proceedings of the Eighth Conference on Machine Learning and Systems (MLSys)*. <https://openreview.net/forum?id=ccjvBkTRRe>

[44] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007